



RAPPORT D'ALTERNANCE

Alternant, développeur de logiciel interne

THIERY Marc

04 Septembre 2023 - 31 Août 2024

Maître d'apprentissage : Monsieur MOULLET
Enseignante référent : Madame POUCKET

Etablissement / Formation : IUT Clermont Auvergne - Département Informatique
Entreprise d'accueil : Sanofi - 45 Chem. de Meteline, 04200 Sisteron

Sommaire

Sommaire.....	1
Glossaire.....	2
Remerciements.....	3
Introduction.....	4
Développement.....	5
Présentation de l'entreprise.....	5
Présentation du site.....	5
Présentation du service.....	7
Présentation du projet.....	8
Gestion du projet.....	9
Développement d'OSIRIS.....	11
Présentation des technologies.....	11
Gestion des données.....	12
Création de recettes.....	13
Récupération des anciennes données.....	13
Gestion de l'éditeur de recette.....	14
Vérification des recettes.....	17
Déplacement de tâche.....	18
Présentation et explication des règles.....	18
Présentation du fonctionnement de l'algorithme de déplacement de tâche.....	20
Problème rencontré et solution apportée.....	23
Gestion des retards.....	24
Création d'un nouveau lot.....	25
Premier positionnement du lot.....	25
Deuxième positionnement du lot.....	26
Affichage des données.....	27
Présentation de l'affichage.....	27
Placement des tâches sur la vue.....	28
Compactage de lot.....	30
Équipement alternatifs.....	32
Gestion en base de données.....	32
Gestion côté affichage.....	32
Vérification pour équipement alternatifs.....	33
Gestion des changements de liens.....	33
Conclusion.....	34
Résumé en anglais.....	35
Bibliographie.....	36

Glossaire

Termes	Définition
EXOP	Excellence opérationnelle.
Procédure stockées	Notion présentée page 13.
MVC	Modèle-Vue-Contrôleur: Modèle d'architecture logicielle qui divise une application en trois composants interconnectés.
MLD	Modèle Logique de Données: Représentation structurée des données d'une base de données.
Vue	Table virtuelle qui permet de visualiser un sous-ensemble des données.
SGBD	Système de Gestion de Base de Données: Logiciel qui permet de créer, gérer et interagir avec des bases de données.
Hitbox	Zone invisible qui détermine si une collision a lieu entre deux objets.
Dénormalisation	Principe de regrouper des données créant de la redondance de données mais améliorant la performance.
Réursive	Une fonction qui s'appelle elle-même.
<div>	Balise HTML utilisée pour regrouper un contenu dans une page web.
Curseur	Permet de parcourir ligne par ligne un ensemble de résultats retournés par une requête SQL.
Trigger	Déclencheur qui s'exécute automatiquement en réponse à un événement spécifique de base de données.
Recette	Succession de tâches permettant de créer un médicament / principe actif
Client lourd	Application installé directement sur l'ordinateur

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué à la réussite de mon alternance et à l'enrichissement de mon expérience professionnelle.

Tout d'abord, je souhaite remercier chaleureusement mon maître d'apprentissage, Monsieur Moullet. Sa disponibilité, sa bienveillance et son expertise ont été essentielles tout au long de mon alternance. Grâce à ses conseils et ses idées, j'ai pu approfondir mes connaissances dans le domaine et acquérir de nouvelles compétences.

Je tiens également à remercier l'ensemble de mon service qui m'a accueilli et accompagné tout au long de cette alternance. Leur accueil chaleureux, leur esprit d'équipe et leur soutien constant ont contribué à créer une ambiance de travail agréable et stimulante. Leurs conseils, leurs expériences partagées et leur bonne humeur ont rendu cette expérience encore plus enrichissante.

Puis, je tiens à exprimer ma reconnaissance envers mon établissement scolaire et mes professeurs qui m'ont permis de réaliser cette alternance. Leur soutien et leurs enseignements m'ont fourni les bases nécessaires pour m'adapter rapidement aux exigences du milieu professionnel.

Je termine en adressant mes sincères remerciements à tous ceux qui ont été impliqués dans mon alternance et qui ont fait de cette expérience une réussite. Votre soutien a été inestimable, et je suis honoré d'avoir pu bénéficier de votre précieuse expertise.

Introduction

Pendant ma troisième année d'études à l'IUT Informatique de Clermont-Ferrand, j'ai eu l'opportunité de réaliser mon alternance au même endroit que celui de mon stage de deuxième année, sur le site Sanofi Sisteron, du 4 septembre 2023 au 31 août 2024. Au cours de cette alternance, ma mission principale était de redévelopper entièrement une application existante, OSIRIS, afin de corriger ses nombreux bugs, d'améliorer sa maintenabilité et de l'optimiser. OSIRIS est une solution web permettant de suivre la production de l'usine. Son objectif est d'automatiser et d'optimiser au maximum les processus, rendant ainsi la planification de la production la plus simple possible. En cas d'imprévu, cette application permet de modifier ce suivi. Elle sert de support à l'ensemble du personnel impliqué dans la production, de près ou de loin. L'application doit être réactive, car elle sera fréquemment utilisée. Elle ne doit comporter aucun bug critique, car cela pourrait perturber la production de toute l'usine, contredisant ainsi son objectif initial. Ainsi, la problématique à laquelle nous faisons face est la suivante : Comment créer une application réactive permettant d'optimiser et d'améliorer le suivi de la production ?

Les objectifs de ce projet sont les suivants : tout d'abord, l'application doit être conçue pour être intuitive et simple d'utilisation, car elle sera utilisée par un large éventail de profils d'utilisateurs. Les temps de calcul visibles pour l'utilisateur doivent être quasi inexistants, car c'était un des problèmes majeurs de l'ancienne version. L'application ne doit contenir aucun bug, surtout pas dans les parties critiques. Enfin, ce projet doit être facilement maintenable, car il est prévu que l'application évolue avec l'ajout de nouvelles fonctionnalités. Si l'application fonctionne correctement sur le site de Sisteron, elle pourra être déployée sur d'autres sites au niveau national et international.

Nous commencerons par une présentation du site Sanofi Sisteron et du service dans lequel je travaille. Par la suite, nous verrons une présentation plus détaillée de l'application. Ensuite, nous examinerons la gestion du projet. Enfin, nous analyserons le développement en mettant l'accent sur les points les plus intéressants permettant ainsi de remplir les différents objectifs fixés.

Développement

Présentation de l'entreprise

Dans cette première partie du développement, nous allons présenter l'entreprise Sanofi dans son ensemble, puis parler plus spécifiquement du site situé à Sisteron, avant d'examiner en détail le service où j'ai effectué mon stage.

Sanofi est l'une des plus grandes entreprises pharmaceutiques mondiales, engagée dans la recherche, le développement, la fabrication et la commercialisation de produits pharmaceutiques. Fondée en 1973, la société a son siège en France et opère à l'échelle mondiale, avec des activités dans plus de 170 pays. Sanofi se concentre sur plusieurs domaines thérapeutiques clés tels que les maladies cardiovasculaires, les maladies métaboliques, les maladies rares, l'oncologie, les vaccins et les médicaments en vente libre.

Sanofi se positionne comme une entreprise intégrée, couvrant toutes les étapes du développement et de la commercialisation des médicaments, de la recherche préclinique à la production en passant par les essais cliniques. La société propose une vaste gamme de produits pharmaceutiques, comprenant des médicaments innovants et des traitements essentiels pour répondre aux besoins de santé de millions de personnes à travers le monde.

Présentation du site

J'ai donc réalisé mon alternance au sein du site de production situé à Sisteron, dans la région Provence-Alpes-Côte d'Azur. Ce site appartient à la filiale Sanofi Chimie ([cf. annexe 1, site de Sisteron](#)).

Le site de Sisteron a été créé en 1917 pour répondre aux besoins de défense nationale par la société Allais et Forge. Il a ensuite été utilisé pour la production de poudre pendant la Seconde Guerre mondiale. En 1973, le site est devenu membre de l'entité Sanofi, mais ce n'est qu'en 1993 que la production de médicaments a commencé. En 1986, le site de Sisteron est devenu membre de la filiale Sanofi Chimie, et depuis lors, il se développe au même rythme que l'entité et compte actuellement plus de 600 employés.



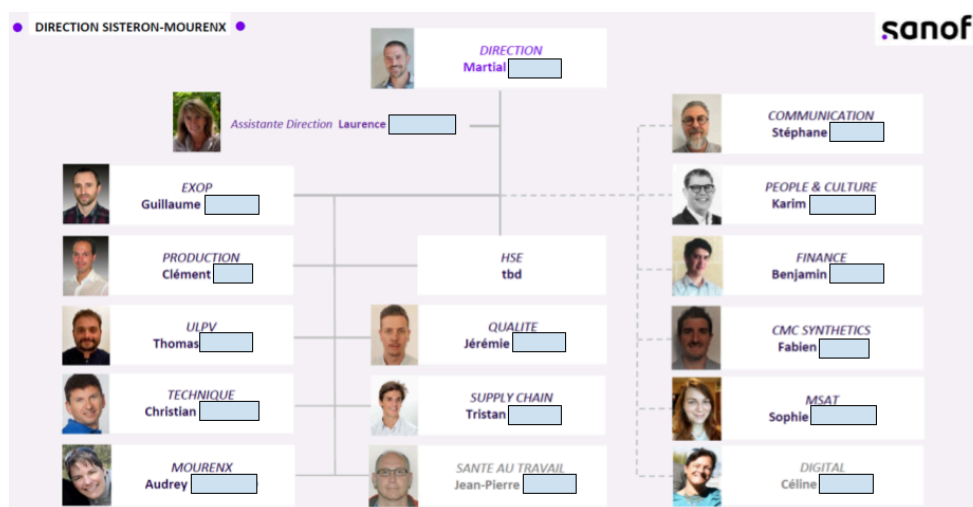
Photo panoramique de l'usine Sanofi Chimie à Sisteron; Source: Sanofi.fr

Au sein du site, deux principales activités sont menées :

La Production : Cette activité vise à fabriquer les principes actifs nécessaires aux usines de conditionnement de l'entité. Elle implique la mise en œuvre de procédés de fabrication rigoureux et de contrôles de qualité pour assurer la production efficace et conforme aux normes.

La Recherche et Développement des procédés (R&D) : Cette activité a pour objectif de développer et d'améliorer les procédés de fabrication des molécules issues de la recherche, en les adaptant aux grandes lignes de production à l'échelle industrielle. La R&D cherche à optimiser les procédés existants, à explorer de nouvelles techniques et technologies, et à garantir la sécurité, la qualité et l'efficacité des produits.

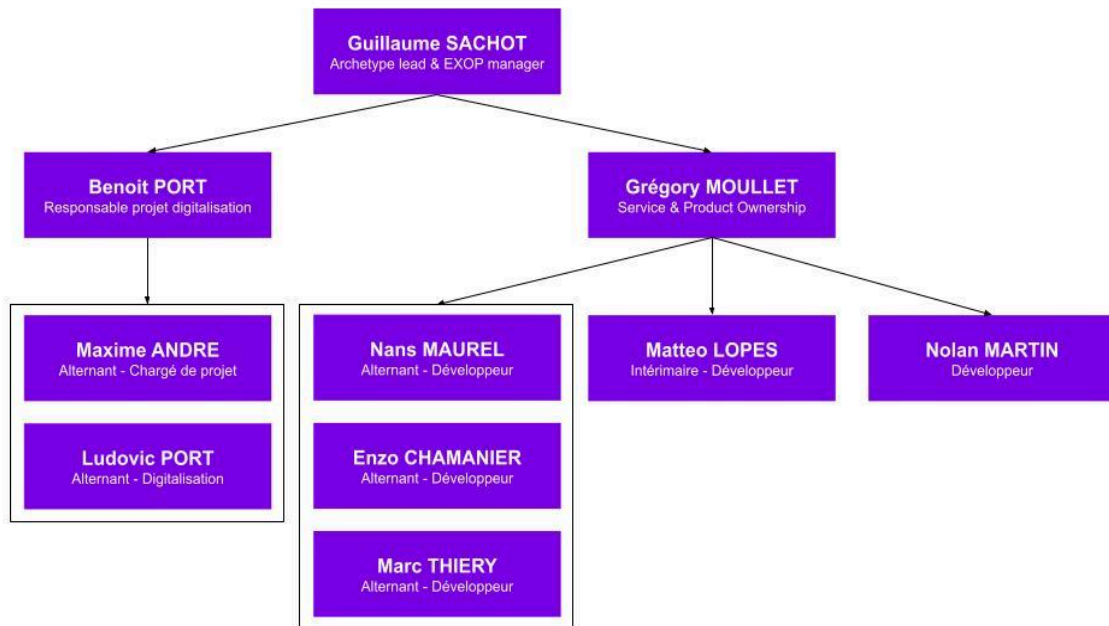
Pour assurer le bon fonctionnement de l'usine, différents services sont présents. Voici les responsables de chaque service, représentés sous forme d'organigramme :



Organigramme direction Sisteron-Mourenx; Source: document interne

Présentation du service

Le service dans lequel je réalise mon alternance est l'Excellence Opérationnelle (EXOP), dirigé par Guillaume Sachot. Ce service est important au sein de l'entreprise, car il est en relation avec la majorité des autres services. Il se divise en deux parties distinctes. Nous allons donc nous pencher sur l'équipe au sein de laquelle on m'a affecté



Organigramme EXOP Digitalisation; Source: document créé

J'ai travaillé au sein de la partie "Digitalisation" d'EXOP. La mission de ce service est de développer des applications et des outils en réponse aux besoins exprimés par les autres services de Sanofi Sisteron. C'est dans ce service que sont centralisées toutes les propositions de digitalisation provenant de l'ensemble de l'usine. L'équipe est principalement composée d'alternants, chacun étant responsable d'un ou plusieurs projets. Deux ingénieurs, Gregory Moullet et Benoît Port, encadrent cette équipe d'alternants. Gregory Moullet, ingénieur en génie industriel spécialisé en mécatronique, a été mon maître d'apprentissage et celui qui m'a accueilli au sein de l'équipe.

Présentation du projet

Comme mentionné dans l'introduction, OSIRIS est un projet ayant un existant déjà bien établi. Il s'agit d'un client lourd, actuellement déployé sur deux sites : Aramon et Sisteron. Le logiciel est pleinement fonctionnel sur le site d'Aramon, où les recettes de production sont moins complexes, ce qui limite le nombre de combinaisons de cas possibles. En revanche, à Sisteron, les exigences sont plus élevées en raison de la complexité des recettes. Cela a entraîné des problèmes imprévus, perturbant le suivi de la production et provoquant de nombreux bugs, ce qui a motivé le développement de cette nouvelle version du projet sur laquelle je travaille.

En plus de cela, Sisteron a des besoins spécifiques, comme la gestion d'équipements alternatifs ou de compactage, deux fonctionnalités qui seront détaillées ultérieurement. Outre la gestion du planning de production, d'autres fonctionnalités sont nécessaires pour rendre le projet aussi complet que possible. L'application doit permettre la création de nouvelles recettes de production, ainsi que la modification des recettes existantes, tout en gérant un système de version. Elle doit également assurer la gestion du nombre d'opérateurs requis à chaque instant, afin de déterminer les ressources humaines nécessaires pour la production. Cela inclut la mise en place d'un système d'équipes, de rotation des équipes, ainsi que la gestion des congés et des présences.

Après avoir brièvement présenté le projet et sa genèse et les objectifs global en introduction, concentrons-nous sur les autres objectifs de cette mission. Ils peuvent être divisés en deux parties : les objectifs du service et mes objectifs personnels.

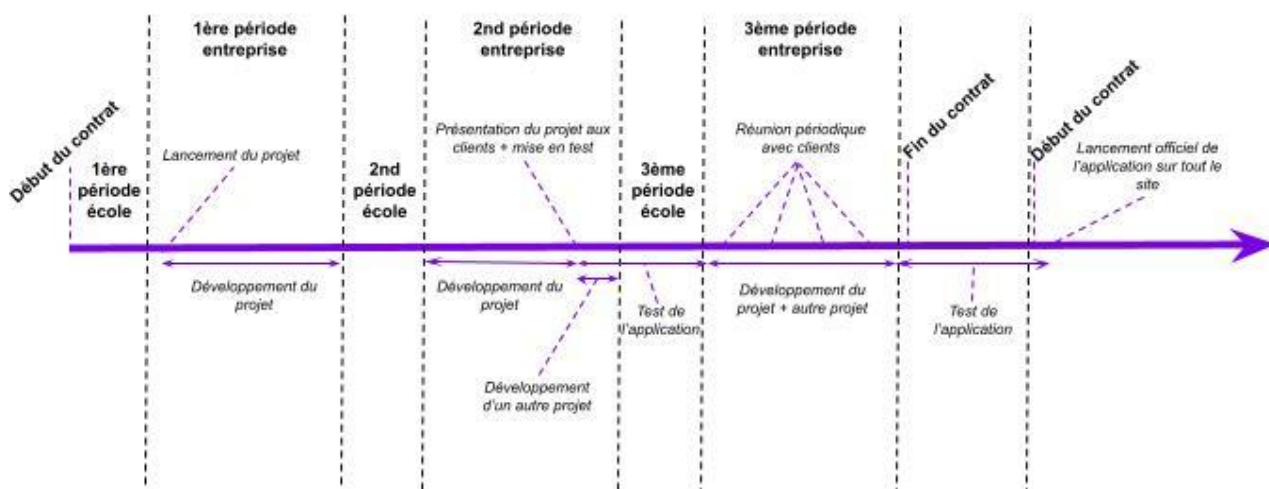
Concernant les objectifs du service, il est essentiel que le code soit lisible, car d'autres développeurs seront amenés à l'utiliser. Par exemple, un membre de l'équipe a rejoint le projet pour réaliser une refonte graphique. Le projet doit n'utiliser que des technologies couramment maîtrisées par les développeurs pour minimiser le temps de formation nécessaire. La plupart des algorithmes doivent être gérés en SQL sous forme de procédures stockées. En effet, mon maître d'apprentissage étant data analyste et maîtrisant parfaitement SQL, il pourra intervenir plus facilement en cas de problème. De plus, les procédures stockées pourront être réutilisées sur d'autres projets.

En ce qui concerne mes objectifs personnels, il est primordial pour moi de remplir les objectifs mentionnés précédemment tout en fournissant un code maintenable et documenté, afin qu'il puisse être repris par quelqu'un d'autre. De plus, je souhaite renforcer mes compétences en web, surtout en gestion de données, qui est l'aspect principale de ce projet.

Gestion du projet

Il est important de connaître le client avant d'aborder la gestion du projet. Il s'agit du service de production de Sanofi Sisteron. Ce service est réparti en plusieurs bâtiments de production, chacun étant géré par une ou des personnes responsables de la planification pour son bâtiment spécifique. Les utilisateurs finaux de l'application seront tous ces responsables de tous les bâtiments, mais pour l'instant, deux bâtiments testeront l'application en avance.

Au sein du service d'Excellence Opérationnelle, aucune méthode de travail n'est imposée, ce qui permet à chacun d'organiser la gestion de son projet selon ses préférences. En tant qu'alternant travaillant de manière autonome, j'ai structuré mon organisation en fonction de mes besoins. Chaque matin, mon maître d'apprentissage et moi nous réunissons pour planifier les tâches du jour et évaluer si d'autres priorités devaient être traitées avant. Ces réunions quotidiennes me permettent de suivre l'avancement du projet et de réajuster les tâches restantes. Ce suivi régulier évite également de mal comprendre une fonctionnalité à développer, ce qui ferait perdre du temps inutilement.



Frise chronologique des dates importantes; Source: document créé

Expliquons la gestion de projet de mon alternance et surtout du projet présenté, grâce à la frise chronologique ci-dessus. Pendant la première période de mon alternance, le projet de la nouvelle version d'OSIRIS a été lancé avec mon maître d'apprentissage. Lors de cette réunion de lancement, nous avons élaboré le cahier des charges et priorisé les tâches à accomplir pendant cette période. À ce stade, je n'utilisais pas encore d'outils ou de méthodes spécifiques pour gérer ces tâches.

Durant la deuxième période, j'ai continué à avancer sur le projet pour le rendre fonctionnel et utilisable. Vers le dernier tiers de cette période, mon maître d'apprentissage, estimant que le projet avait suffisamment progressé, a organisé la première réunion avec les clients pour que je puisse présenter le projet et recueillir leurs différents avis. Les clients, très satisfaits, ont proposé de lancer la phase de test de l'application en l'utilisant pour suivre la production de leurs bâtiments. Nous avons alors décidé d'organiser des réunions périodiques toutes les deux semaines, durant lesquelles les clients feraient un compte rendu de leur utilisation de l'application, des bugs rencontrés, et des améliorations possibles. De mon côté, je faisais le point sur les corrections et améliorations apportées entre les réunions, ainsi que sur les tâches prévues jusqu'à la prochaine rencontre.

À la fin de cette deuxième période et durant la troisième, j'ai ajusté ma manière de conduire le projet. Avec la mise en place de réunions périodiques, il devenait nécessaire d'organiser plus efficacement l'avancement du projet. Pour ce faire, j'ai intégré des outils comme un Backlog et un Kanban. Ces outils m'ont été très utiles pour faire suivre toutes les tâches accomplies entre deux réunions et pour ajouter les nouvelles tâches à réaliser. Enfin après avoir discuté avec un alternant chargé de projet dans mon service, nous nous sommes mis d'accord pour qu'il s'occupe de l'aspect gestion de projet d'OSIRIS ainsi que de la partie formation pour former les opérateurs qui utiliseront OSIRIS.

Développement d'OSIRIS

Présentation des technologies



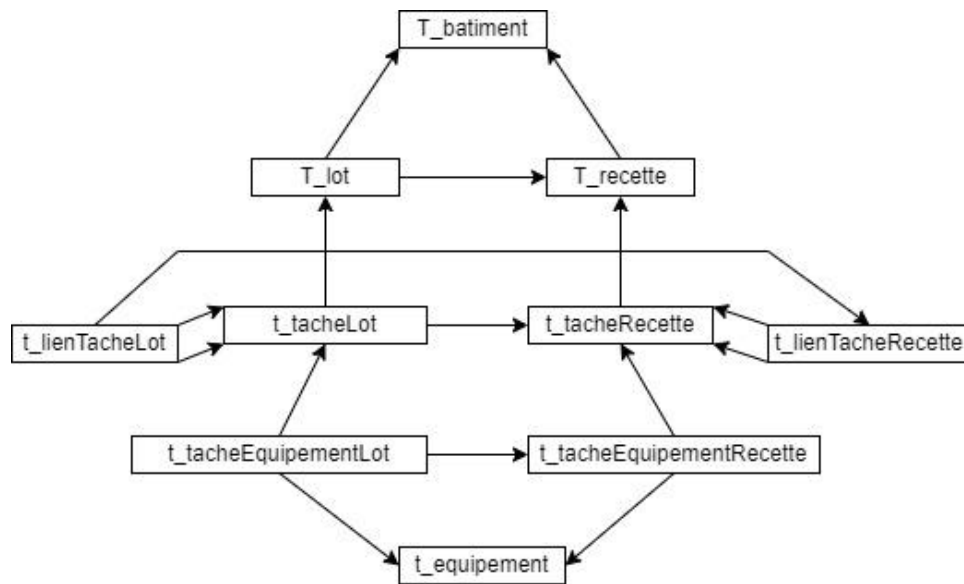
Pour ce projet, les technologies ont été choisies en fonction de leur popularité et de la probabilité qu'un autre développeur les maîtrise déjà, et JavaScript ainsi que PHP répondent parfaitement à ce critère.

JavaScript, utilisé côté client, est essentiel pour rendre les pages web dynamiques et interactives, des qualités indispensables pour ce projet. De son côté, PHP, un langage côté serveur, facilite la communication avec la base de données de manière simple et efficace, ce qui en fait un choix judicieux pour gérer la logique serveur et les interactions avec les données. En plus de cela, j'ai eu la chance d'avoir pu étudier ces deux langages au cours de mes années d'études à mon IUT, me permettant de respecter certaines règles et d'architecture, où l'on peut penser à MVC pour PHP.

En ce qui concerne la gestion des données, il était nécessaire d'utiliser un système de gestion de bases de données. En effet, nous avons besoin d'une solution nous permettant de visualiser facilement ces données. Le service EXOP utilise largement les bases de données *via* SQL Server Management Studio (SSMS), il était donc logique d'utiliser cet outil. Pour le présenter brièvement, SQL Server Management Studio est un outil de gestion et d'administration des bases de données Microsoft SQL Server. Il s'agit d'une application Windows offrant une interface graphique conviviale qui permet aux développeurs et aux administrateurs de gérer facilement les bases de données SQL Server. SSMS propose des fonctionnalités telles qu'un éditeur de requêtes, un explorateur d'objets, la gestion des autorisations ainsi que l'importation et l'exportation de données.



Gestion des données



Conception principale de la partie planning d'OSIRIS sans les attributs. Source : document créé

Afin d'expliquer l'organisation et les relations entre les données, comparons la production de principe actif chez Sanofi à la création d'un gâteau. Il existe de nombreuses recettes à réaliser (**T_recette**), et chaque recette est composée de plusieurs tâches, telles que casser les œufs ou préchauffer le four (**T_tacheRecette**). Certaines de ces tâches dépendent d'autres, par exemple, pour battre les œufs, il faut d'abord les avoir cassés (**T_lienTacheRecette**). Il est également possible de lancer plusieurs tâches en parallèle, comme préchauffer le four tout en préparant la pâte. De plus, certaines tâches peuvent nécessiter un délai d'attente, comme laisser reposer la pâte. Ces tâches doivent être effectuées dans des équipements spécifiques, tels qu'un four ou un saladier (**T_equipement**), et pour une même tâche, plusieurs équipements peuvent être utilisés en même temps, comme différents moules à gâteau (**T_tacheEquipeementRecette**).

Dans OSIRIS, tout ce processus est automatisé. Le MLD présenté ci-dessus montre plusieurs relations entre les tables "T_...recette" et "T_...lot". Lorsqu'une recette est lancée en production, ce n'est pas directement la recette qui apparaît dans le planning ni qui est modifiable. À la place, une copie des données de la recette est créée dans un lot de production. Cette approche a plusieurs avantages. Reprenons l'exemple du gâteau: on peut estimer à l'avance la durée totale de la recette en l'absence d'imprévus, mais il est impossible de prédire précisément quand chaque tâche commencera et se terminera, ou si tous les équipements seront fonctionnels. Il est donc crucial de pouvoir modifier ces données en temps réel. Si les

modifications étaient appliquées directement sur la recette originale, cela affecterait l'ensemble de la production utilisant cette recette. En créant une copie pour chaque lot, on peut donc modifier les données spécifiques à ce lot sans impacter les autres, assurant ainsi une gestion plus flexible et précise de la production.

Création de recettes

Récupération des anciennes données

Afin d'avoir des données pour tester l'application, nous avons besoin de recettes. Étant donné que ces recettes ont déjà été informatisées dans la version précédente d'OSIRIS, il serait judicieux de récupérer ces données existantes et de les reformater pour qu'elles soient compatibles avec la nouvelle conception de cette version. Cependant, il est important de noter que les bases de données des deux versions d'OSIRIS ne se trouvent pas sur le même serveur. Pour cela, il existe une solution permettant de transférer des données d'un serveur à un autre : l'utilisation d'un système de vue. SQL Server Management Studio (SSMS) offre la possibilité de lier deux SGBD entre eux, ce qui permet de créer une vue capable d'accéder aux données d'une base de données et de renvoyer les résultats à l'autre base.

Maintenant que nous avons accès aux données, il ne reste plus qu'à copier les résultats dans nos différentes tables, le tout étant géré par une procédure stockée. Il est pertinent de s'attarder sur ce qu'est une procédure stockée et sur les raisons de son utilisation; sachant que cette technologie a été largement employée tout au long du développement d'OSIRIS. Une procédure stockée en SQL est un ensemble de requêtes SQL précompilées, stockées dans la base de données. Elle permet d'automatiser des opérations répétitives ou complexes en exécutant un bloc de code SQL *via* une seule commande sous un nom spécifique. Cela signifie que dans notre code PHP, il n'y a jamais de requêtes SQL volumineuses, seulement une instruction invoquant la procédure stockée avec les paramètres appropriés. Le code devient ainsi plus lisible, car il suffit de faire appel à la procédure en utilisant un nom suffisamment explicite. De plus, si un autre projet nécessite de copier les données de la même manière, il pourra directement exécuter cette procédure pour obtenir le résultat souhaité.

```
EXEC copyDataFromOSIRISV1;
```

```
CREATE OR ALTER PROCEDURE copyDataFromOSIRISV1
AS
BEGIN
    INSERT INTO T_recette (id_recette, nom, groupe_gamme,
                          numArticle, type_idR, bat_id, bat_nom, groupe_id,
                          active, standardQty)
    SELECT vp_id, vp_nom, vp_groupe_gamme, vp_numArticle, type_id, bat_id,
          bat_nom, gr_id, vp_active, vp_standardqty
    FROM V_vp_old
    WHERE vp_active = 1 AND vp_disabled = 0;
    ....
END;
```

Extrait de la procédure stockée permettant de copier les données provenant d'une vue dans les tables d'OSIRIS . Source : Code OSIRIS

Nos recettes sont maintenant copiées et intégrées dans notre base de données. Il est donc nécessaire de mettre en place des fonctionnalités permettant de les visualiser, modifier ou supprimer.

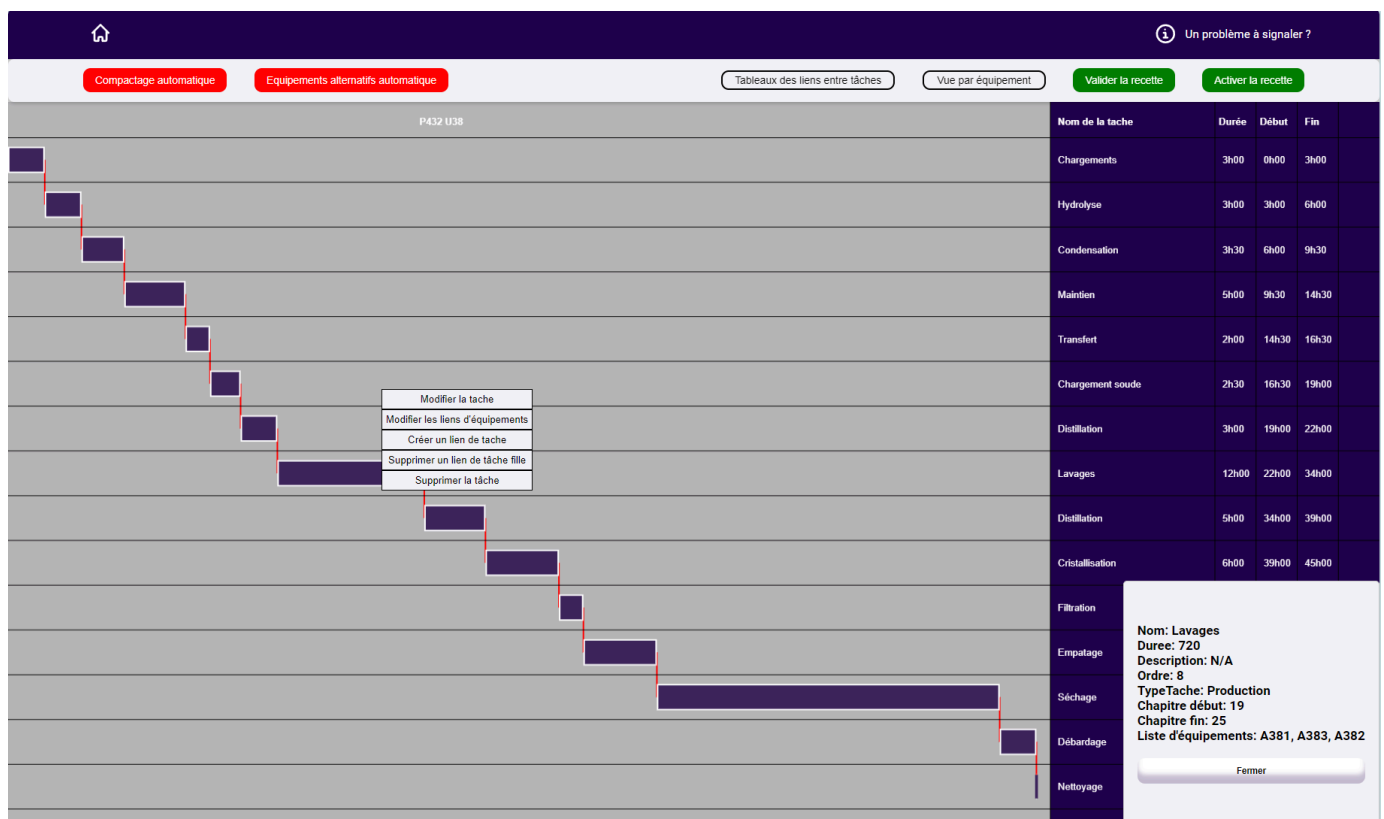
Il est important de noter que les recettes que nous modifions ici peuvent potentiellement être utilisées par des lots en production. Même si nous créons une copie des données, certains algorithmes, qui seront présentés ultérieurement, nécessitent parfois de récupérer les données de la recette originale. Par conséquent, lorsqu'une recette est sélectionnée et qu'elle est actuellement utilisée par un lot, il n'est pas possible de la modifier directement. À la place, un autre bouton permet de créer une nouvelle version de la recette, qui sera simplement une copie de la recette originale avec un incrément de 1 dans le champ "version" stocké dans la table T_recette.

Gestion de l'éditeur de recette

À ce stade du projet, les besoins n'étaient pas clairement définis, et mon maître d'apprentissage n'était pas disponible. J'ai donc dû adapter les outils utilisés pour réaliser cette fonctionnalité afin de minimiser les risques de devoir revenir en arrière. Pour permettre la visualisation et la modification des recettes, j'ai choisi d'utiliser la balise <canvas> de HTML5. Cette balise permet de créer des graphiques 2D et des visualisations complexes avec un contrôle précis sur chaque pixel grâce à JavaScript. Elle offre des performances supérieures à celles de ses alternatives

comme <svg> pour les mises à jour en temps réel, garantissant un rendu rapide et fluide. Cela en fait un choix particulièrement adapté aux applications graphiques interactives nécessitant une personnalisation fine.

Bien que ce choix présente de nombreux avantages, il comporte également des inconvénients. Contrairement à d'autres technologies, les éléments dessinés sur un <canvas> ne peuvent pas directement recevoir d'événements (comme les clics de souris). Il est donc nécessaire de gérer ces événements à un niveau global et de calculer manuellement les interactions, ce qui ajoute une couche de complexité supplémentaire.



Exemple de visualisation de l'éditeur de recette avec événement lorsqu'on clique sur une tâche.

Source : Capture d'écran d'OSIRIS

Avec la capture d'écran ci-dessus, on observe nos différentes tâches représentées par des rectangles. Il est relativement simple d'ajouter un événement pour détecter un clic de souris sur ces tâches, car en définissant leur position et leur taille, il suffit de mettre en place un système de hitbox. On peut notamment voir ce qu'il se passe lorsqu'on clique sur une tâche où une multitude d'actions sont possibles en regardant la capture d'écran.

Mais cela se complexifie pour la gestion d'événements au niveau des liens entre tâches. En effet, il est possible de réaliser des actions sur les liens comme le modifier ou supprimer. On connaît seulement deux points, le point sur la fin de la tâche mère et le début de la tâche fille. Afin de pouvoir déterminer la distance entre la souris et un lien afin de réaliser des actions sur ce dernier, il nous faut calculer en premier l'équation de la droite avec la pente.

Soit (x_1, y_1) et (x_2, y_2) , les deux points connus de la droite :

Calcul de la pente : $m = \frac{y_2 - y_1}{x_2 - x_1}$

Équation de la droite: $y - y_1 = m(x - x_1)$

Équation sous forme général $(Ax + Bx + C) = 0$:

$$(y_2 - y_1)x - (x_2 - x_1)y + ((x_2 - x_1)y_1 - (y_2 - y_1)x_1) = 0$$

Soit: $A = y_2 - y_1$

$$B = -(x_2 - x_1)$$

$$C = (x_2 - x_1)y_1 - (y_2 - y_1)x_1$$

A présent, on peut calculer la distance entre notre souris et la droite grâce à la formule, Soit (x_3, y_3) la position de la souris:

$$d = \frac{|Ax_3 + By_3 + C|}{\sqrt{A^2 + B^2}}$$

Dans ce cas précis, nous cherchons à déterminer la distance entre un point et un segment, plutôt qu'une droite. La solution la plus simple trouvée consiste à gérer ce problème en utilisant la coordonnée y des deux points. En comparant les positions des deux points, il est facile de déterminer lequel est au-dessus de l'autre par rapport à l'origine 0. Ensuite, il suffit de vérifier si la position y de la souris est inférieure à celle du point le plus haut, supérieure à celle du point le plus bas, et suffisamment proche du segment pour conclure que la souris est bien sur le lien. Cela permet d'autoriser ou non des actions supplémentaires.

Cela étant dit, on peut se demander si l'utilisation d'une autre balise, qui permettrait de gérer les événements de souris directement, n'aurait pas été une meilleure solution en termes de performance, surtout en considérant qu'il peut y avoir des dizaines de liens à gérer. Il reste à explorer comment ce type de problème est abordé avec ces autres balises.

Vérification des recettes

Maintenant que nos recettes sont entièrement éditables, nous devons mettre en place un algorithme de validation pour vérifier une multitude de critères afin de s'assurer que chaque recette est conforme aux règles de production. Pour rappel, après analyse des problèmes rencontrés dans la première version d'OSIRIS, qui a échoué à Sisteron, il est apparu que l'un des principaux soucis était les erreurs illogiques commises lors de la conception des recettes du côté production. Ces recettes, une fois intégrées dans le planning, provoquaient de nombreux bugs lors des recalages de temps. Cette phase du projet est donc l'une des plus critiques.

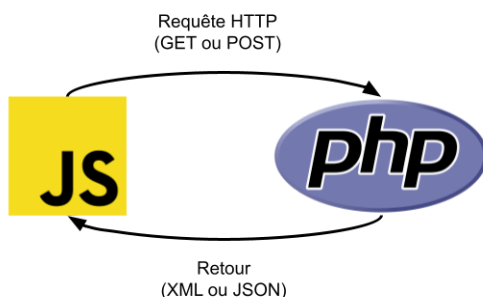
Voici la liste des cas qui sont gérés par l'algorithme:

- ✓ La tâche d'ordre 1 ne possède pas de parent
- ✓ Toutes les tâches ont au moins un équipement
- ✓ Toutes les tâches ont au moins un lien parent
- ✓ La tâche Ordre 1 commence en premier
- ✓ La tâche Ordre max se termine en dernier
- ✓ Aucune tâche ne se superpose
- ✓ Pas de dépendance circulaire
- ✓ La recette est valide !

Confirmer

L'approche par promesses permet de gérer en enchaînant chaque vérification de manière séquentielle. Cela signifie que chaque critère est vérifié uniquement si la validation précédente a été réussie. Pour l'optimisation de temps, on traite en priorité les cas les plus probables et les plus simples d'erreur de recette. Ainsi, les algorithmes plus gourmands, comme la vérification des dépendances circulaires (qui consiste à s'assurer qu'une tâche ancêtre d'un parent ne dépend pas d'une tâche fille),

ne sont exécutés que si tous les tests précédents passent.



Par ailleurs, chaque vérification utilise une requête AJAX (Asynchronous JavaScript and XML), permettant d'effectuer des requêtes asynchrones depuis JavaScript vers le serveur PHP. Cela permet d'interroger la base de données, de renvoyer les réponses, et de gérer les messages d'erreur, le tout sans recharger la

page web, rendant ainsi l'application très dynamique. Une fois la vérification de la recette terminée, le système permet de refermer le pop-up et d'appliquer les modifications sans avoir à recharger la page.

Une fois nos recettes validées, elles peuvent être activées pour être lancées en production. Mais avant de lancer un lot en production, il est essentiel de présenter le point central d'OSIRIS : l'algorithme de déplacement de tâches. Cet algorithme permet de positionner correctement chaque tâche d'un lot de production, en respectant non seulement les règles de la recette, mais aussi celles des autres lots et des interdépendances entre les lots.

Déplacement de tâche

Présentation et explication des règles

Pour commencer, il est important de rappeler les différentes règles à prendre en compte pour expliquer l'algorithme de déplacement des tâches.

Une tâche de lot, pour se positionner correctement, nécessite plusieurs informations. Tout d'abord, elle possède une date de début, une durée, et une date de fin. Il est à noter que la date de fin peut être facilement calculée en additionnant la date de début et la durée. Cependant, l'idée de stocker cette donnée permet d'éviter de la recalculer à chaque requête. Bien que nos données occupent plus d'espace, elles sont rendues plus rapidement. De plus, une tâche de lot dispose d'un attribut 'originDuree', qui stocke la durée initiale de la tâche. Cela permet de savoir si la tâche a pris plus de temps que prévu, quelle était sa durée d'origine. De la même manière, la durée initiale pourrait être récupérée en recherchant la tâche de recette associée, sachant qu'une tâche de lot est une copie de celle-ci. Cependant, cela nécessiterait une jointure pour associer les deux données, un temps de calcul que l'on peut éviter en stockant directement la durée d'origine dans un autre attribut de la tâche de lot. C'est le principe de la dénormalisation.

Un autre attribut intéressant à mentionner est la notion de verrou. Pour rappel, les tâches sont reliées entre elles par des liens de dépendance, ce qui signifie qu'elles se positionnent en fonction de leurs parents. Il peut arriver qu'une tâche de lot soit déplacée si elle doit commencer plus tard que prévu. Par conséquent, les tâches dépendantes de cette tâche (les tâches filles) devront s'adapter aux nouvelles données de leur parent. Par exemple, si le parent commence plus tard, les tâches filles commenceront également plus tard, et cela se propage de manière récursive.



C'est ici que la notion de verrou intervient. Pour chaque tâche de lot, il est possible de définir si elle est verrouillée ou non. Si une tâche est verrouillée, elle ne sera pas affectée par un changement dans la tâche mère si celle-ci se termine plus tôt que prévu. En revanche, si la tâche mère se termine plus tard que prévu, la tâche verrouillée sera tout de même poussée, comme les autres.

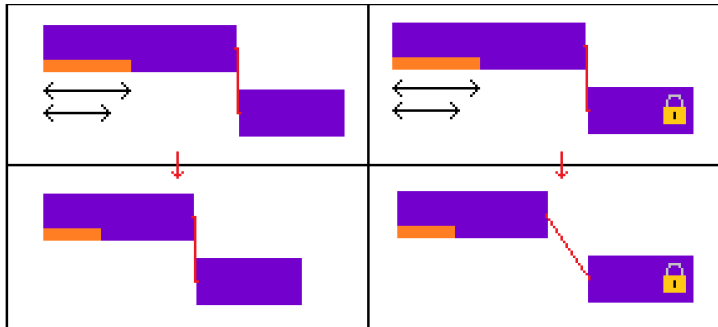
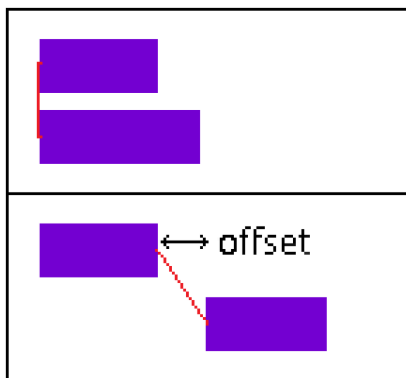


Schéma montrant la différence entre la réduction de durée de la tâche mère et le comportement de la tâche fille selon si elle est verrouillée ou non.

Source : Document créé

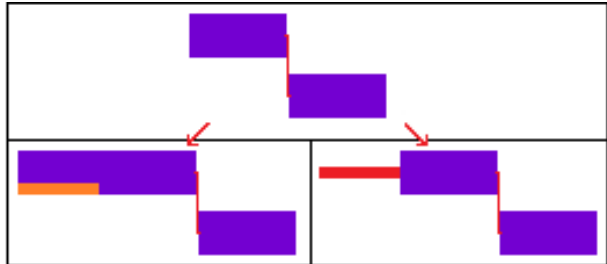
L'utilité de cette fonctionnalité réside dans le fait qu'en production, il peut y avoir des temps de pause, par exemple lorsque certains bâtiments ne produisent pas le week-end, et où l'optimisation du temps de fabrication du lot ne doit pas nécessairement être maximale.

En ce qui concerne les liens (dépendances) entre tâches, deux attributs sont à prendre en compte : le type de lien et l'offset. Tout d'abord, il existe deux types de liens : le lien 'DD' et le lien 'FD'. Le lien 'DD' (Début-Début) indique que deux tâches doivent commencer en même temps, comme illustré dans la première case, tandis que le lien 'FD' (Fin-Début) indique que la tâche fille commence à la fin de la tâche mère, comme dans la deuxième case. L'offset, quant à lui, représente un temps d'attente minimum entre deux tâches. Cet attribut est rarement utilisé, mais peut être crucial pour certaines recettes.



Il est important de noter que dans ce contexte, les liens semblent simples car ils impliquent une tâche mère et une tâche fille. Cependant, en réalité, une tâche peut avoir plusieurs tâches filles et dépendre de plusieurs tâches mères. Il est donc nécessaire de calculer, à chaque fois, sur quelle tâche mère se baser pour ne pas positionner la tâche de manière illogique. De plus, une tâche peut parfois être affectée à plusieurs équipements simultanément, ce qui complique davantage les calculs.

Un dernier point à aborder avant de présenter l'algorithme concerne la notion de retard. Dans OSIRIS, il est possible de modifier le planning en déplaçant des tâches de lots, ce qui met à jour toutes les tâches dépendantes. Cependant, il est également important de pouvoir retracer ces modifications visuellement. Il existe deux façons de modifier le planning : la première consiste à déplacer une tâche pour la faire commencer plus tard, ce qui déclenche l'apparition d'un pop-up de retard où



l'on doit indiquer la raison du retard (case de droite). La deuxième consiste à créer un retard de durée, c'est-à-dire qu'après avoir commencé la production, la tâche de lot prend plus de temps que prévu, nécessitant ainsi une extension de la

durée de la tâche (case de gauche). Cela permet de générer des statistiques sur les retards fréquents, leur moment d'apparition, afin de réfléchir aux moyens de les éviter. Nous reviendrons plus en détail sur la gestion des retards après la présentation de l'algorithme de déplacement des tâches.

Présentation du fonctionnement de l'algorithme de déplacement de tâche

```

1 PROCÉDURE deplaceTache(id_tacheParam, newDateDebut, newDuree)
2
3 Création d'une liste tâches avec id_tacheParam comme premier élément
4 Création compteur = 0
5 Récupération origineDuree, verrou pour id_tacheParam
6
7 tant que le nombre de tâches > 0 et compteur < 1000
8   tacheActuelle = la première tâche dans tâches
9   compteur + 1
10
11 si tacheActuelle n'est pas = à id_tacheParam
12   on récupère newDuree, origineDuree, newDateDebut, verrou pour tacheActuelle
13
14 nbMere <- calcul nombre de parents et données associés
15 si nbMere > 1
16   parcours de nbMere
17     si typeLien = 'FD'
18       dateStockageM = fin tâche mère, offset, typeLien
19     sinon si typeLien = 'DD'
20       dateStockageM = début tâche mère, offset, typeLien
21       dateStockage <- offset + dateStockageM
22       si dateStockage > maxDate
23         maxDate = dateStockage, offset, typeLien
24   sinon si nbMere = 1
25     on met à jour maxDate, offset, typeLien avec les données du lien
26   sinon maxDate <- newDateDebut
27

```

```

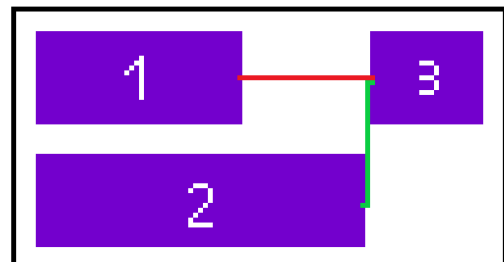
28 calcul dureeRetardDuree si la tâche a un retard de durée, on récupère la durée
29
30 nouvelleDateDebut <- offset + maxDate
31 si newDateDebut n'est pas = nouvelleDateDebut ou tacheActuelle = id_tacheParam ou il
   existe un dureeRetardDuree
32   si pas de verrou ou verrou et id_tacheParam = tacheActuelle ou
33   verrou et nouvelleDateDebut > newDateDebut
34     mise à jour debutReel pour tacheActuelle avec nouvelleDateDebut
35     si il n'y a pas de dureeRetardDuree
36       mise à jour finReel <- debutReel + newDuree et newDuree pour tacheActuelle
37     sinon
38       newDuree <- dureeRetardDuree + origineDuree
39       mise à jour finReel <- debutReel + newDuree et newDuree pour tacheActuelle
40     ajout des tâches filles de tacheActuelle dans tâches
41
42 on supprime tacheActuelle de tâches
43 réinitialisation des variables nécessaires

```

Pseudo code simplifié de l'algorithme de déplacement de tâche.

Source : document crée

1. **Ligne 1** : On crée notre procédure stockée nommée `deplaceTache`, prenant en paramètres l'identifiant de la tâche à modifier, sa nouvelle date de début, et sa nouvelle durée.
2. **Ligne 3** : On crée une liste destinée à stocker les identifiants des tâches qui nécessitent potentiellement d'être modifiées.
3. **Ligne 4** : On crée un compteur qui servira de condition d'arrêt. À chaque itération de la boucle, on incrémente le compteur de 1, et l'algorithme s'arrêtera si l'on dépasse 1000 tâches à déplacer. Cela permet d'éviter de bloquer l'application en cas de déplacement d'une tâche trop loin dans le passé.
4. **Ligne 5** : On récupère certaines données de la tâche passée en paramètre, qui seront utiles pour la suite.
5. **Lignes 7 à 9** : On entre dans une boucle qui parcourt la liste des tâches afin de les modifier si nécessaire. Ensuite, on sélectionne la première tâche de la liste comme tâche actuelle.
6. **Lignes 11 à 12** : Si la tâche actuelle n'est pas celle passée en paramètre, on récupère les données importantes de cette tâche.
7. **Lignes 14 à 26** : Ici, on cherche la tâche mère et le lien associé qui donnent la date maximale pour positionner la tâche actuelle de manière logique. Si la tâche a plusieurs mères (comme dans l'exemple où la tâche 3 possède deux liens parents de type 'FD'), elle doit commencer après



la fin de toutes les mères. On détermine la mère avec la date de fin + offset la plus élevée pour un lien 'FD', ou la date de début pour un lien 'DD'. On stocke les données du résultat, qui incluent la date maximale, l'offset, et le type de lien. S'il n'y a qu'un seul lien, on stocke directement les données du lien. S'il n'y a pas de lien, la date maximale est celle passée en paramètre.

8. **Ligne 28** : On tente de récupérer la durée d'un retard, s'il y en a un, associé à la tâche actuelle. On obtient soit une durée, soit une valeur nulle.
9. **Lignes 30 à 31** : On ajoute la meilleure date mère à l'offset du lien, puis on teste plusieurs cas pour déterminer si la tâche actuelle doit être modifiée. Si la date de début stockée en base diffère de la meilleure date calculée, si la tâche actuelle est celle passée en paramètre, ou s'il existe un retard de durée, on entre dans la condition.
10. **Lignes 32 à 33** : On vérifie ensuite les conditions de la tâche. Si la tâche n'est pas verrouillée, ou si elle est verrouillée mais correspond à la tâche actuelle, ou encore si elle est verrouillée mais que la meilleure date est supérieure à la date de début de la tâche, alors cette dernière doit être modifiée.
11. **Lignes 34 à 39** : On met à jour la date de début de la tâche avec la meilleure date. Ensuite, on vérifie s'il existe un retard de durée. Si ce n'est pas le cas, on met à jour la durée et la date de fin en fonction de la nouvelle date de début et de la durée. Sinon, on combine la durée d'origine avec la durée du retard pour définir la nouvelle durée de la tâche, puis on met à jour la durée et la date de fin.
12. **Ligne 40** : Si la tâche devait potentiellement être modifiée, on ajoute ses tâches filles à la liste des tâches à traiter. Cela réduit considérablement le temps de calcul de l'algorithme en ne traitant que les tâches nécessaires.
13. **Lignes 43 à 44** : Enfin, on supprime la tâche actuelle de la liste des tâches et on réinitialise les variables nécessaires.

L'objectif principal de cet algorithme est bien sûr qu'il fonctionne, mais surtout d'assurer un temps d'exécution extrêmement rapide. Mon maître d'apprentissage m'avait fixé comme objectif de mettre à jour 200 tâches en moins d'une seconde. Objectif atteint, puisque l'algorithme boucle en 0,002 seconde, permettant ainsi de traiter 200 tâches en 0,4 seconde. Ce qui comble l'un des principaux défauts d'OSIRIS V1.

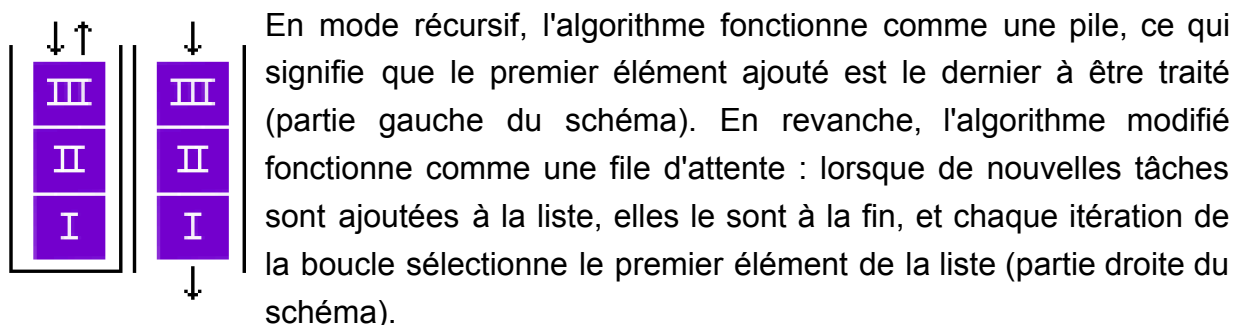
Problème rencontré et solution apportée

Il y a un point important qu'il serait intéressant de traiter lors du développement de cet algorithme.

Maximum stored procedure, function, trigger, or view nesting level exceeded (limit 32)

À l'origine, l'algorithme était conçu de manière récursive, ce qui signifie qu'il s'appelait lui-même en passant les paramètres des tâches filles, sans nécessiter de boucle englobant la procédure. Cependant, cette approche a révélé une limite lorsque j'ai commencé à tester des déplacements sur de vrais lots de production. Une erreur est apparue, indiquant que la limite de 32 niveaux de récursion avait été atteinte. Ce bug n'avait pas été détecté lors des tests initiaux, car mes lots de test étaient relativement petits.

Il n'est pas possible de résoudre ce problème en augmentant simplement cette limite, car cela nécessiterait l'approbation de Sanofi au niveau national pour autoriser une telle modification sur SSMS. J'ai donc dû adopter l'approche non récursive décrite précédemment, qui offre en plus un avantage notable.



L'avantage de cette approche en file d'attente est qu'elle permet de s'assurer que tous les parents d'une tâche sont déjà positionnés avant de traiter cette dernière. Cela évite de devoir déplacer plusieurs fois une tâche fille.

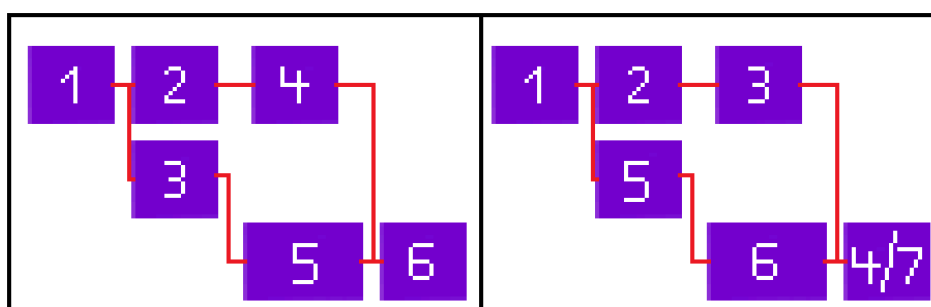


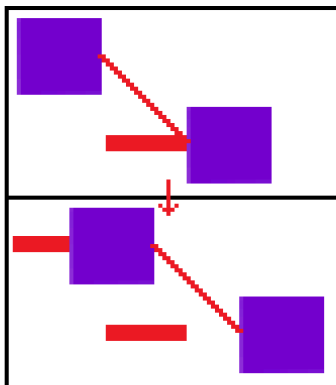
Schéma montrant la différence de parcours de traitement entre une file à gauche et une pile à droite. Source : Document créé

Supposons que l'on déplace la première tâche de manière significative, entraînant ainsi le déplacement de toutes les autres tâches. On peut alors observer comment l'algorithme se comporte différemment selon s'il utilise une file ou une pile. Avec une pile, un déplacement supplémentaire est nécessaire. La raison est qu'à l'étape 4, l'étape 6 n'a pas encore été déplacée et est encore dans le passé, car l'algorithme ne l'a pas encore traitée. Ainsi, l'étape 4 va se positionner en fonction de la date maximale actuelle, c'est-à-dire à la fin de l'étape 3. Puis, lorsque l'étape 6 est enfin déplacée, elle entraîne le déplacement de l'étape 7, qui se positionne à son tour en fonction de la nouvelle date maximale, soit la fin de l'étape 6. En utilisant une file, notre algorithme évite ce déplacement supplémentaire et gagne ainsi en vitesse d'exécution car les deux parents ont déjà été traités.

Gestion des retards

Passons maintenant rapidement à la gestion des retards. Comme mentionné précédemment, un retard est créé lorsqu'une tâche est déplacée (dépassant un écart de 15 minutes). Un retard n'est pas associé à une autre table, mais est traité comme une tâche. Concrètement, lorsqu'un retard est créé, une tâche est générée avec les mêmes attributs qu'une tâche classique, sauf qu'un champ est modifié : le type de tâche, qui devient 'retardTemps' ou 'retardDurée' au lieu de 'production' pour les tâches classiques. Cette tâche n'a pas de parent, mais elle a comme fille la tâche que l'on vient de déplacer. Ainsi, dans l'algorithme, une tâche va également prendre en compte les retards comme parents pour calculer la date maximale. Il existe une différence entre les deux types de retards : un retard de temps est associé à un lien 'FD', tandis qu'un retard de durée est lié à un lien 'DD', mais les deux ont un offset à 0.

Cependant, un bug logique existe. Étant donné qu'un retard n'a pas de parent, car cela n'aurait pas de sens, si une tâche est déplacée avant un retard, les tâches



vont se déplacer pour se réajuster, mais le retard ne suivra pas la tâche associée, car aucun parent ne va la déclencher. Ce cas est défini comme un bug logique car, si un retard est créé pour une tâche, c'est que l'on est en train d'exécuter cette tâche ou sur le point de commencer. Un retard avant un autre retard n'a donc aucun sens. Pour partiellement résoudre ce problème, l'utilisateur a la possibilité de supprimer un retard en cas de mauvaise manipulation.

Création d'un nouveau lot

Notre algorithme de déplacement désormais en place, il devient possible de créer un nouveau lot et de le positionner correctement dans le planning. Cependant, un problème persiste : nos lots ne sont actuellement pas liés entre eux. Il est donc nécessaire d'établir ces liens. Mais avant d'aborder cette étape, faisons un détour pour expliquer comment créer et lancer un lot.

T_ordre_fab	
of_nom	
of_quantite	
of_debut_plannifie	
of_numlot	
vp_nom	
vp_id	
bat_id	
of_color	
isAutoCompact	
isAutoAlternate	

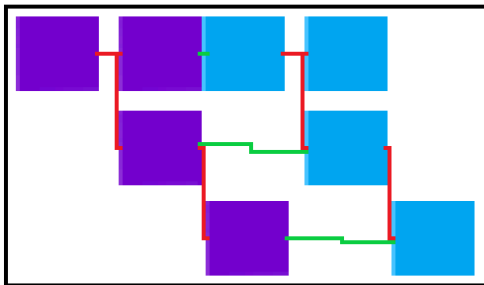
Il existe deux méthodes pour créer un lot : soit manuellement, soit en récupérant les ordres de fabrication *via* SAP. Un ordre de fabrication définit les informations nécessaires au lancement du lot en production. Par exemple, on y trouve le nom du lot, la date planifiée de son lancement, l'identifiant du bâtiment concerné, ainsi que, le plus important, l'identifiant de la recette de fabrication associée. Une fois l'ordre de fabrication créé ou importé, l'utilisateur peut procéder au lancement du lot.

Premier positionnement du lot

Lors de l'exécution de l'algorithme de création de lots, la première étape consiste à copier les données de la recette indiquée depuis l'ordre de fabrication vers les tables de lots, comme expliqué précédemment. Passons maintenant à la gestion des liens entre les lots. Pour cela, nous lançons une première fois l'algorithme de déplacement de tâches, en prenant comme paramètre la première tâche du lot à ajouter. Il est assez simple de récupérer la première tâche du lot grâce à l'attribut 'ordre' présent dans les tâches de lots, qui indique leur position de traitement au sein du lot. Cet attribut est d'ailleurs utilisé lors de la création des recettes pour offrir une vue descendante des recettes. C'est également la raison pour laquelle l'ordre 1 est vérifié dans l'algorithme présenté précédemment (page 16). Nous identifions donc la tâche avec l'ordre 1 pour l'identifiant de la recette spécifiée, en utilisant la date planifiée prévue dans l'ordre de fabrication comme début de la tâche et la durée de la tâche de recette associée.

Deuxième positionnement du lot

Une fois le lot positionné, il reste à utiliser un curseur pour itérer sur chaque équipement utilisé dans le lot. Pour chaque équipement, nous identifions la tâche de ce lot ayant la date de début la plus petite, ainsi que la tâche d'un autre lot se terminant le plus tard pour le même équipement. Ces deux tâches identifiées, nous créons des liens entre elles, de type 'FD' avec un offset de 0. Ensuite, on relance l'algorithme de déplacement de tâches une seconde fois, afin d'adapter les tâches en fonction des nouveaux liens parents issus du lot précédent.



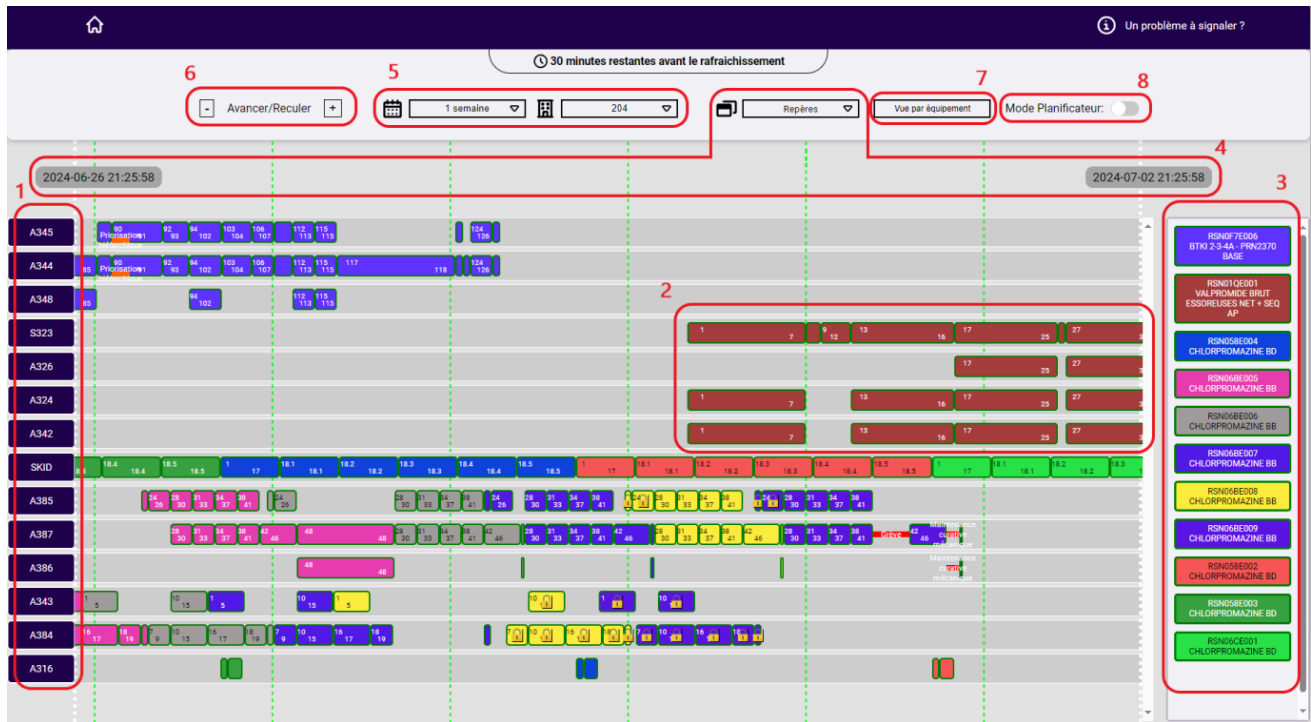
En se référant au schéma ci-contre, on pourrait se demander pourquoi ne pas simplement lier la première tâche d'ordre 1 du lot avec la tâche d'un autre lot ayant la date de fin maximale. Il y a deux raisons à cela. Premièrement, il est possible de créer des retards. Si un retard est significativement appliqué à l'une des deux tâches

du bas (représentées en violet), ces tâches ne pousseraient pas les tâches situées sur la même ligne (en bleu). Deuxièmement, une règle de production chez Sanofi stipule que les lots de production ne doivent pas se croiser : sur un même équipement, il est interdit de passer d'une tâche du lot 1 à une tâche du lot 2, puis de revenir à une tâche du lot 1. En reliant donc toutes les tâches sur tous les équipements concernés, on empêche ce cas de se produire.

Maintenant que tous nos lots sont bien positionnés, il serait pertinent de voir à quoi ressemble le planning dans OSIRIS, comment les différents algorithmes sont lancés, et comment les tâches sont affichées à l'écran.

Affichage des données

Présentation de l'affichage



Vue du planning de production d'OSIRIS V2. Source : Capture d'écran d'OSIRIS

Voici à quoi ressemble la partie principale d'OSIRIS, la partie planning. Il nous faut tout d'abord présenter les différentes parties de l'affichage afin de mettre plus de contexte pour ensuite voir comment placer les tâches de lots.

La partie principale d'OSIRIS est son planning, qui est crucial pour la gestion des tâches de production. Avant de plonger dans le processus de placement des tâches de lots, il est important de comprendre les différentes sections de l'interface pour mieux contextualiser ce qui a été présenté précédemment.

1. **Liste des équipements** : Chaque ligne correspond à un équipement utilisé par les tâches de lots. Un équipement est affiché dès qu'une tâche l'utilise.
2. **Affichage des tâches de lots**: Chaque lot de production est représenté avec ses tâches correspondantes. Les tâches verrouillées et celles avec des retards sont également visibles plus bas. Toutes les tâches sont interactives : en cliquant dessus, on peut consulter leurs données et les déplacer par glisser-déposer (drag and drop).

3. **Liste des lots** : Cette section affiche tous les lots ayant au moins une tâche visible dans le planning. On peut cliquer sur un lot pour effectuer des actions ou consulter ses données. De plus, en survolant un lot avec la souris, les tâches associées sont mises en surbrillance dans le planning.
4. **Barre de temps** : Cette barre indique la date de début et de fin de l'affichage. Si la date actuelle se situe entre ces deux dates, elle est affichée, et le curseur de la souris indique en permanence la date exacte à l'endroit pointé. Le bouton de repère permet d'afficher des barres verticales marquant les heures, les postes, les jours ou les semaines. Le repère approprié est automatiquement sélectionné en fonction de la durée d'affichage demandée.
5. **Sélection du bâtiment et de la durée** : Cette section permet de choisir le bâtiment à afficher et de définir la durée d'affichage. Il est également possible de personnaliser les dates affichées.
6. **Boutons de déplacement de la date d'affichage** : Ces boutons permettent de décaler la date d'affichage en avant ou en arrière. Le temps de décalage est automatiquement calculé en fonction de la durée d'affichage.
7. **Vue triée des équipements** : Ici, on peut choisir l'ordre d'affichage des équipements, selon un tri personnalisé.
8. **Mode planificateur** : Ce mode, accessible uniquement avec des droits spécifiques, permet de déplacer des tâches sans qu'il soit nécessaire de spécifier une cause de retard.

Placement des tâches sur la vue

Passons à présent au placement des tâches. Tout d'abord, il n'y a pas de balise spécifique qui a été utilisée dans cette partie, seulement des éléments `<div>`. Au départ, PHP nous retourne la liste des tâches à afficher ainsi que les identifiants des équipements auxquels elles appartiennent avec leurs noms. De plus, les deux dates d'affichage demandées, c'est-à-dire la date de début et de fin, sont également fournies.

La première étape consiste à construire un objet qui va contenir l'identifiant et le nom de chaque équipement, ainsi que la liste des tâches associées à cet équipement sur les dates données. Une fois ces objets construits, il ne reste plus qu'à itérer sur cette liste pour placer d'abord la `<div>` qui accueillera les tâches. Cette `<div>` est décomposée en deux sous `<div>`, l'une à gauche pour afficher le nom de l'équipement et l'autre pour servir de conteneur aux futures tâches.

Passons maintenant au placement des tâches. Ce qui nous intéresse ici est de positionner le début et la durée de chaque tâche de manière proportionnelle à l'échelle de temps définie. Pour cela, la première chose à faire est de calculer le temps que représente l'affichage, c'est-à-dire la différence entre la date de fin et la date de début, que l'on exprime en secondes pour simplifier les calculs. Cette durée représente 100% de la longueur du conteneur. Ensuite, on prend chaque tâche, et on effectue un produit en croix. Sachant que la durée d'affichage en secondes vaut 100%, on prend la date de début de la tâche, convertie en secondes, puis on soustrait la date de début d'affichage en secondes pour obtenir une valeur qui, une fois le produit en croix effectué, nous donne le pourcentage du conteneur où placer la tâche.

*(début de la Tâche – début affichage) * 100 / durée différence = ? où se placer%*

On fait presque de même pour la durée de la tâche:

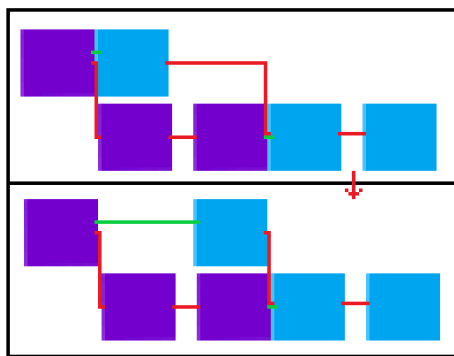
*durée de la tâche * 100 / durée différence = ? taille que prend la durée %*

Maintenant que l'on connaît nos placements, il ne reste plus qu'à créer notre `<div>` et à définir les attributs nécessaires. L'attribut `left` indiquera où la tâche commence, et `width` définit la durée de la tâche. Pour les déplacements de tâches, lorsque l'utilisateur termine le drag and drop, une fonction se déclenche pour calculer, à partir des coordonnées de la souris et des dimensions du conteneur, le nombre de secondes à ajouter au début de la tâche ou à la durée de celle-ci. Cette valeur est ensuite convertie en date pour lancer l'algorithme de déplacement de tâche. À noter qu'il est possible de récupérer les données de la position d'une tâche placée à l'écran en utilisant une fonction de JavaScript: `getBoundingClientRect()`, qui retourne les coordonnées `x` et `y` de la tâche ainsi que sa hauteur et sa largeur. Ce sera utile pour récupérer les paramètres pour l'algorithme de déplacement de tâche.

Passons à la présentation de deux fonctionnalités avancées intéressantes qui méritent d'être expliquées.

Compactage de lot

Le principe du compactage de lot est assez simple. Actuellement, à chaque placement de lot ou déplacement de tâches, celles qui doivent être déplacées sont positionnées de manière à être produites le plus rapidement possible. Cependant, dans la réalité, en production, ce n'est pas toujours nécessaire. Prenons l'exemple d'une recette de gâteau : techniquement, rien ne nous empêche de casser les œufs trois heures avant de les mélanger, mais ce n'est pas vraiment utile. C'est pourquoi il est important de pouvoir compacter un lot de production, en déplaçant les tâches dans l'autre sens, c'est-à-dire en les rapprochant du début de la tâche fille la plus proche. Il est actuellement possible de simuler un compactage en créant différents retards sur les tâches, mais cela prend du temps, surcharge la vue inutilement, et étant donné le nombre de lots nécessitant cette logique de compactage, il est judicieux de développer une fonctionnalité dédiée pour cela.



Voici un exemple de compactage. Sur le schéma, les liens de lots sont représentés en rouge, tandis que les liens entre lots sont en vert. Dans la partie inférieure de l'image, on observe le lancement du compactage sur la première tâche en bleu. Cette tâche, à condition qu'aucune autre tâche ne l'empêche, se repositionne de manière à se rapprocher de sa tâche fille ayant la date de début la plus proche, tout en tenant compte d'un éventuel décalage ou "offset" s'il en existe un.

L'algorithme de compactage, toujours en procédure stockée, ressemble beaucoup à l'algorithme de déplacement de tâche, mais il se limite aux tâches d'un même lot et parcourt ce lot en commençant par la ou les dernières tâches pour remonter jusqu'à la tâche d'ordre 1.

Pour identifier les dernières tâches d'un lot donné, on récupère celles qui n'ont pas de filles du même lot, en se concentrant uniquement sur les liens de recettes pour ignorer les liens entre lots. Comme les liens de tâche de recette sont copiés lors de la duplication, cela permet de ne travailler qu'avec les liens internes au lot en vérifiant s'il existe une correspondance côté recette.

Maintenant que le compactage des lots est possible, mon maître d'apprentissage a demandé d'intégrer une option de compactage automatique. Pour cela, on ajoute un

champ 'isAutoCompact' dans la recette, qui est ensuite copié dans les lots associés, avec la possibilité de modifier le compactage automatique une fois en production. Pour gérer cette automatisation, j'avais initialement proposé l'utilisation d'un trigger SQL, mais cette approche a été rejetée car elle compliquerait la reprise du projet par un autre développeur, qui ne penserait pas forcément à vérifier l'existence de triggers.

J'ai donc opté pour une autre solution. Lorsque l'utilisateur demande la liste des tâches sur une période donnée, on passe par une procédure stockée pour formater les données avant de les renvoyer. On enregistre la date de la requête et on la compare à une date stockée dans une table. Si la date stockée a plus de trois heures d'écart avec la date actuelle, on la remplace par la nouvelle date, puis on lance une procédure stockée qui récupère tous les lots encore en cours et ayant le compactage automatique activé. Ainsi, pour chaque lot, l'algorithme de compactage est exécuté, et ce dernier est compacté. Une fois une tâche compactée, elle est verrouillée pour éviter qu'elle ne soit prise en compte dans l'algorithme de déplacement de tâche en cas de modification ultérieure.

Equipement alternatifs

Passons à la dernière fonctionnalité intéressante à présenter. Le nom de cette fonctionnalité est assez explicite : elle permet à l'utilisateur de modifier l'équipement d'une tâche.

Cette fonctionnalité implique plusieurs points, notamment la gestion en base de données pour réassocier la tâche à un autre équipement, la vérification de la cohérence du déplacement, ainsi que la suppression et la création de nouveaux liens.

Gestion en base de données

Pour commencer, on utilise la table 'T_tacheEquipementLot' qui permet de gérer les liens d'équipement/tâche et maintenant d'équipements alternatifs. Elle contient l'identifiant de la tâche et celui de l'équipement associé. Deux champs importants sont 'ordre' et 'priorité'. Le champ 'ordre' gère les cas où une tâche utilise plusieurs équipements simultanément, en assignant un ordre à chaque équipement. Le champ 'priorité' intervient lorsqu'une tâche dispose d'un équipement alternatif : une ligne supplémentaire est ajoutée avec une priorité à 2, tandis que l'équipement actif conserve une priorité à 1. Tous les algorithmes précédemment présentés utilisent uniquement les liens tâche/équipement avec une priorité à 1.

T_tacheEquipementLot	
id_tacheEquipementL	
ordre	
priorite	
id_tacheR	
id_equipement	
id_tacheEquipementR	
id_tacheL	

Gestion côté affichage

Pour permettre à l'utilisateur de changer d'équipement, une option est ajoutée en cliquant sur une tâche. Cela permet de sélectionner un nouvel équipement, où l'équipement actif est affiché en vert. Trois options sont ensuite proposées : modifier uniquement la tâche sélectionnée, toutes les tâches du même lot, ou toutes les tâches suivantes de ce lot vers le nouvel équipement sélectionné. Cela se traduit simplement par une requête différente dans la procédure stockée associée.

Modification équipement alternatif

Bac cata AG4401 AG47 FIR3101 HOC3101 HOC501 Hydrogénateur AG44 LF14301
LF14321 REA2101 REA2201 REA2301 REA2401

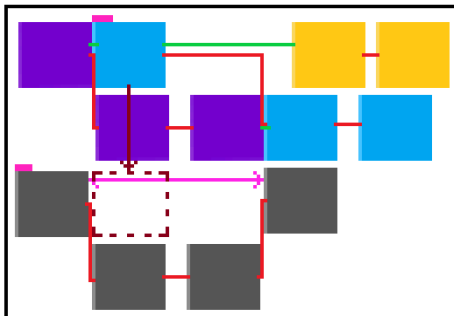
Méthode d'application sur l'équipement

La tâche Les tâches du lot Les tâches suivantes

Confirmer Annuler

Vérification pour équipement alternatifs

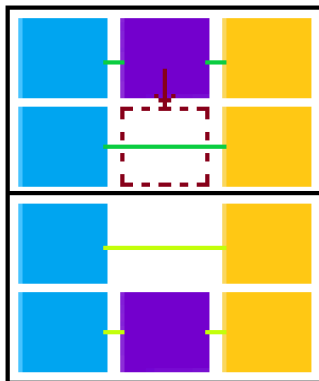
Deux vérifications sont ensuite effectuées : d'abord, on s'assure qu'il n'y a pas de tâche existante dans la zone où l'on souhaite déplacer la ou les tâches, ce qui se fait par une requête. Ensuite, on vérifie que les lots ne se croisent pas. Pour ce faire, on identifie la tâche qui précède les potentielles nouvelles tâches sur le nouvel équipement, ainsi que le lot associés et la date de début. On compare ensuite les dates avec celles des tâches à déplacer. Si toutes les conditions sont réunies, le déplacement des tâches peut être effectué.



Pour résumer ce qui a été dit précédemment, dans le cas où l'on souhaite déplacer la première tâche bleue vers l'emplacement rouge bordeaux, la première étape consiste à vérifier s'il y a suffisamment de place, représentée par la double flèche rose. Ensuite, on compare les deux premières tâches des lots, symbolisées par un petit trait rose au-dessus d'elles. Si la logique

d'anti-croisement est respectée, on peut alors procéder à l'étape suivante du déplacement.

Gestion des changements de liens



La dernière étape de l'algorithme concerne la gestion des liens entre les tâches lorsqu'un déplacement est effectué. Il peut être nécessaire de supprimer jusqu'à trois anciens liens et d'en créer trois nouveaux. Sur la première case en haut du schéma, on observe les différents liens entre lots en vert. Si l'on déplace une tâche, par exemple la tâche violette vers la deuxième ligne, la première action consiste à supprimer les liens verts existants, ensuite, on déplace la tâche, puis on crée les nouveaux liens, représentés en vert clair dans la partie inférieure du schéma. Ce scénario représente le cas le plus complexe, mais l'algorithme est conçu pour s'adapter à différents cas, en gérant les suppressions et créations de liens en fonction du nombre de liens parent ou enfant présents.

Grâce à l'ajout de cette fonctionnalité, j'ai également pu intégrer une autre option permettant de supprimer un lot en production, en réutilisant la logique de suppression de liens développée pour la gestion des équipements alternatifs.

Conclusion

En conclusion, ce projet a été initié en réponse aux nombreuses demandes des différents clients internes de Sanofi Sisteron. L'application existante présentait des bugs et des temps d'attente trop longs, ce qui suscitait de la frustration chez les utilisateurs et entraînait une diminution de son utilisation. Cependant, au début du lancement d'OSIRIS V1, les clients ont exprimé une grande satisfaction, car cette application apportait une réelle valeur ajoutée à leur quotidien professionnel. C'est à partir de ce succès temporaire, que le service EXOP a décidé de développer OSIRIS V2, le projet auquel j'ai été affecté.

OSIRIS est une application de support à la production, conçue pour accompagner les utilisateurs dans le suivi de la production et leur permettre de modifier ce suivi à tout moment. Toutes les modifications sont traçables, offrant ainsi la possibilité d'analyser et d'optimiser les processus de production. OSIRIS facilite également la planification en automatisant et en optimisant les processus, rendant la gestion de la production plus simple pour les utilisateurs. De plus, elle propose une multitude de fonctionnalités personnalisables, telles que le compactage de lots et la gestion des équipements alternatifs.

Les principaux objectifs du projet étaient de corriger les défauts de la première version d'OSIRIS, notamment en éliminant ou en réduisant les temps d'attente et en résolvant les problèmes liés à des cas non prévus dans les entrées utilisateur, l'origine de la plupart des bugs d'OSIRIS V1. L'application devait également être maintenable par le service, ce qui impliquait une réflexion approfondie sur l'utilisation des technologies choisies. Sur le plan personnel, mes objectifs étaient de développer mes compétences dans les domaines sur lesquels j'ai travaillé et de livrer un projet fonctionnel à la fin de mon année d'alternance.

Dans l'ensemble, le projet a été un succès, atteignant les différents objectifs fixés. Il a le potentiel d'être utilisé sur tout le site de Sisteron et pourrait même être déployé sur d'autres sites de production en France, si l'on en croit les retours positifs des clients et de mon maître d'apprentissage. Le projet suscite également de l'intérêt à l'international, notamment de la part d'un site Sanofi à Singapour. OSIRIS sera officiellement lancé à mon retour en mi-octobre, et je continuerai à travailler sur ce projet lors de mes trois prochaines années d'alternance chez Sanofi Sisteron. Bien que le projet ait déjà beaucoup évolué, il reste encore beaucoup à faire, car OSIRIS continuera à évoluer en fonction des besoins des clients.

Résumé en anglais

OSIRIS is a web application designed to assist users in monitoring Sanofi Sisteron's production and allowing them to modify this tracking at any time. All changes are traceable, providing the ability to analyze and optimize production processes. OSIRIS also simplifies planning by automating and optimizing processes, making production management easier for users.

First, in the development section, we introduce Sanofi on a global level, followed by the Sisteron site and the department where I was assigned during my internship and apprenticeship last year.

Next, we present the key aspects of the first version of OSIRIS and explain why a new version of this project needs to be developed, to which I was assigned. A brief presentation of OSIRIS project management explains its evolution.

In the main section on OSIRIS development, only the relevant parts of the application's development are presented.

First, a presentation of the technologies used, along with the reasons for their selection, is provided. The main data design of OSIRIS is then discussed. Following this, an explanation is given on how to create and retrieve recipes in the application for modification. The main OSIRIS algorithm is then introduced, which enables the movement of a task or multiple tasks and the automatic optimization of production. Next, the process of creating a new production batch and integrating it into the rest of the production is shown. The display section illustrates how data is organized and calculated to optimize task placement. Finally, two advanced features of OSIRIS are presented, allowing for more customization of OSIRIS tracking. The first is batch compaction, which allows a batch to be compressed and completed in a shorter period. The second is alternative equipment, which explains how to change one or more tasks to different equipment and the implications of these changes.

Bibliographie

Informations sur Sanofi:

<https://fr.wikipedia.org/wiki/Sanofi>

Informations sur le site Sanofi Sisteron:

<https://www.sanofi.fr/fr/notre-entreprise/nos-sites-en-france/le-site-de-production-de-sisteron>

<https://www.sanofi.fr/fr/media/communiques-et-dossiers-de-presse/2021/sanofi-investit-plus-de-60-millions-d-euros-sur-son-site-de-sisteron>

Principe MVC:

<https://dyma.fr/blog/introduction-au-mvc-avec-php/>

Lier deux serveurs SQL serveur:

<https://learn.microsoft.com/fr-fr/sql/relational-databases/linked-servers/create-linked-servers-sql-server-database-engine?view=sql-server-ver16>

Principe d'une vue:

<https://www.pairform.fr/les-vues-sql.html>

Créer une procédure stockée:

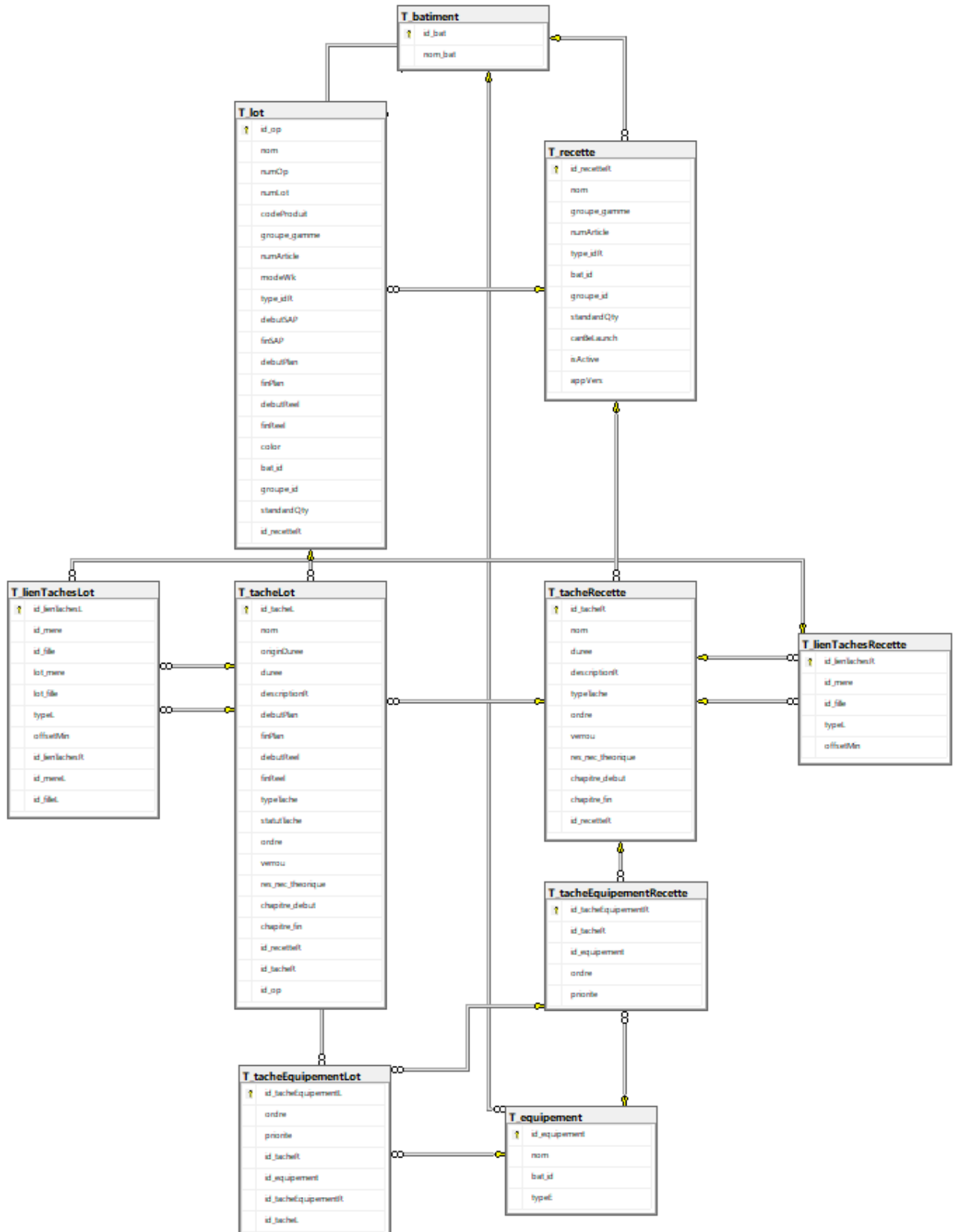
<https://learn.microsoft.com/fr-fr/sql/relational-databases/stored-procedures/create-a-stored-procedure?view=sql-server-ver16>

Annexe 1 : Site de Sisteron



Situation géographique du site Sanofi Chimie Sisteron. Source : Google maps

Annexe 2 : MLD partie planning



Capture d'écran de la conception de la partie planning d'OSIRIS. Source : interne

Annexe 3 : Développement de l'équation de droite

$$y - y_1 = m(x - x_1)$$

On remplace m :

$$y - y_1 = \frac{y_2 - y_1}{x_2 - x_1}(x - x_1)$$

On multiplie de chaque côté par $(x_2 - x_1)$:

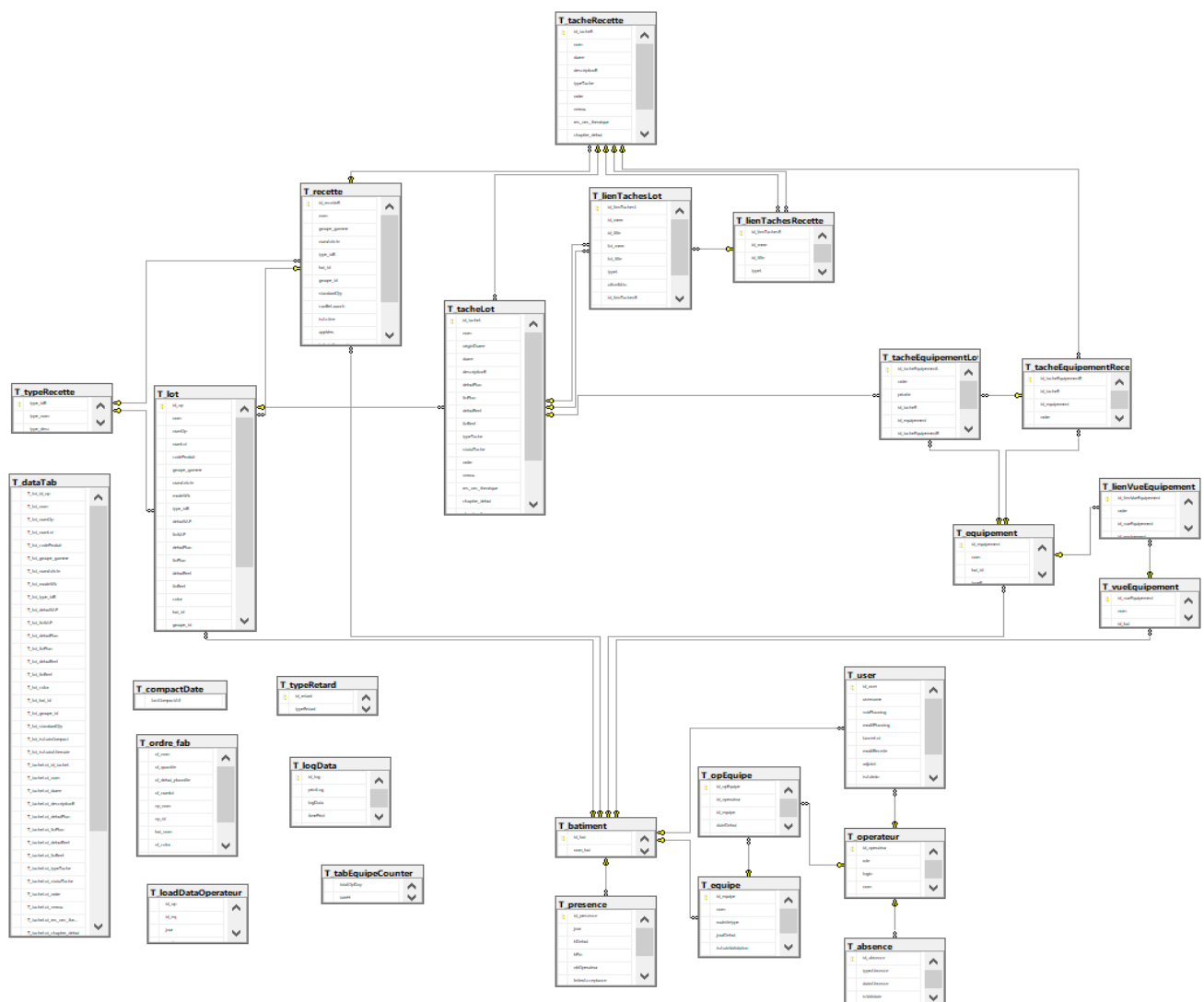
$$(x_2 - x_1)(y - y_1) = (y_2 - y_1)(x - x_1)$$

On développe:

$$(x_2 - x_1)y - (x_2 - x_1)y_1 = (y_2 - y_1)x - (y_2 - y_1)x_1$$

$$(y_2 - y_1)x - (x_2 - x_1)y + ((x_2 - x_1)y_1 - (y_2 - y_1)x_1) = 0$$

Annexe 4 : MLD complet



Capture d'écran de la conception d'OSIRIS. Source : interne